DAY 17 | HANDS-ON ACTIVITIES

DevOps and Cloud Fundamentals: Problem Statements

Seven problems on DevOps, DevSecOps and cloud concepts with a local CI simulator. Azure tasks use the web portal and Azure DevOps in the browser; delete any resources afterwards.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Map the Loop

Level: Warm-up | **Concepts:** DevOps loop

For your SmartDesk app, write what happens at each stage of the DevOps loop: plan, code, build and test, release, operate and monitor.

Sample run / expected result

```
Plan: user story "filter tickets by priority" on the board
Code: feature branch + pull request
...
```

Hints

- Be concrete: name the tool and the artefact at each stage.

Problem 2: Delivery or Deployment?

Level: Easy | **Concepts:** CI, continuous delivery, continuous deployment

For each team, say which practice they use: (a) tests run on every push, release monthly by hand; (b) every green build waits for a manager's click; (c) every green build goes live.

Sample run / expected result

```
(a) CI only
(b) Continuous delivery
(c) Continuous deployment
```

Hints

- Ask: is there a human approval step?

Problem 3: Cloud Model and Migration Path

Level: Easy | **Concepts:** IaaS, PaaS, SaaS, rehost to replace

Pick a model and a migration path for: a legacy ticketing server needing a custom Windows driver; a FastAPI service; company email; an old reporting app the team wants to rewrite; a database nobody wants to patch.

Sample run / expected result

```
Legacy server: IaaS, rehost
FastAPI: PaaS (App Service), refactor
Email: SaaS, replace
```

Hints

- The more you must control, the further left you go.

Problem 4: Local CI Simulator

Level: Medium | **Concepts:** pipeline steps, fail fast

Using the notebook, run the CI simulator on the sample project. Introduce a failing test, watch the pipeline stop, then fix it and get a green run.

Sample run / expected result

```
[1/2] Lint         ... OK
[2/2] Unit tests   ... FAILED
Pipeline stopped
```

Hints

- Read the failing test output before changing code.

Problem 5: Scan and Fix NEW (v4)

Level: Medium | **Concepts:** SAST, secret scanning

Run bandit and the notebook's secret scanner on the sample report module. Record each finding (tool, severity, file, line, proposed fix), fix the code, and re-run until both scans are clean.

Sample run / expected result

```
bandit: 4 issues (B404 Low, B105 Low, B602 High, B307 Medium)
secrets: 1 finding in report.py:3
after fixes: bandit -ll 0 issues, secrets 0 findings
```

Hints

- Move the secret to an environment variable.
- Use a list of arguments instead of shell=True.

Problem 6: Write a Pipeline YAML with a Security Stage NEW (v4)

Level: Challenging | **Concepts:** azure-pipelines.yml, stages

Write a pipeline that triggers on main and pull requests, runs on ubuntu-latest, uses Python 3.12, has a Build stage (install, ruff, pytest with JUnit results) and a Security stage (bandit, pip-audit). Validate it with yaml.safe_load.

Sample run / expected result

```
Stages: Build (3 steps), Security (3 steps)
Trigger: ['main']
```

Hints

- PublishTestResults should run even if tests fail: condition: always().

Problem 7: Boards Backlog

Level: Stretch | **Concepts:** Azure Boards, user stories

Write 6 user stories for SmartDesk with acceptance criteria, group them into 2 sprints, and mark one as a bug and one as a security task. If you have Azure DevOps access, enter them in Boards.

Sample run / expected result

```
As an agent, I want to filter tickets by priority so that I can focus on urgent work.
Acceptance: filter shows only matching tickets; clearing it shows all.
```

Hints

- Use the format: As a ..., I want ..., so that ...